

Operating Systems and Concurrency

Lecture 1: Introduction
COMP2007

Dan Marsden
(Chao Chen)
{Chao.Chen, Dan.Marsden}@Nottingham.ac.uk

University Of Nottingham
United Kingdom

2026

What's the course about?

Operating Systems

What's the course about?

Operating Systems

- Fundamental abstraction letting 'normal programs' interact with hardware

What's the course about?

Operating Systems

- Fundamental abstraction letting 'normal programs' interact with hardware
- Aim for extreme efficiency - leading to greater complexity

What's the course about?

Operating Systems

- Fundamental abstraction letting 'normal programs' interact with hardware
- Aim for extreme efficiency - leading to greater complexity
- Evolve to exploit latest hardware innovations

What's the course about?

Operating Systems

- Fundamental abstraction letting 'normal programs' interact with hardware
- Aim for extreme efficiency - leading to greater complexity
- Evolve to exploit latest hardware innovations
- Understanding operating systems improves us as computer scientists

What's the course about?

Operating Systems

- Fundamental abstraction letting 'normal programs' interact with hardware
- Aim for extreme efficiency - leading to greater complexity
- Evolve to exploit latest hardware innovations
- Understanding operating systems improves us as computer scientists
 - Need to understand what the OS is doing to get good performance

What's the course about?

Operating Systems

- Fundamental abstraction letting 'normal programs' interact with hardware
- Aim for extreme efficiency - leading to greater complexity
- Evolve to exploit latest hardware innovations
- Understanding operating systems improves us as computer scientists
 - Need to understand what the OS is doing to get good performance
 - Understanding a complex program like an OS is extremely instructive

What's the course about?

Concurrency

- Running multiple pieces of code “at the same time”

What's the course about?

Concurrency

- Running multiple pieces of code “at the same time”
- Vital to make good use of hardware if you're living post 2000

What's the course about?

Concurrency

- Running multiple pieces of code “at the same time”
- Vital to make good use of hardware if you're living post 2000
- Challenging to program - combinatorial explosion of possible executions

What's the course about?

Concurrency

- Running multiple pieces of code “at the same time”
- Vital to make good use of hardware if you're living post 2000
- Challenging to program - combinatorial explosion of possible executions
- Lots of open theoretical and practical questions

What's the course about?

Concurrency

- Running multiple pieces of code “at the same time”
- Vital to make good use of hardware if you're living post 2000
- Challenging to program - combinatorial explosion of possible executions
- Lots of open theoretical and practical questions
- Operating system critical to providing concurrency

Goals

What Will be Covered?

- Introduction to the **fundamental concepts, key principles** and **internals** of **operating systems** and **concurrency**
- Understand how **application programs interact with the operating system**
- Basic understanding of writing **concurrent and parallel code** and **OS principles** related to concurrency

Lectures

When and Where?

- **Lectures** are on Monday, Thursday, and Friday (**first four weeks**)
- **Slides** will be online before lectures, and may be updated afterwards
- Lecture recordings will be available
- There are also **labs** on Wednesday, starting in the second week.

Lectures

Subjects We Will Discuss

Subject	#Lectures	By
Introduction to operating systems/computer design	3	DM
Processes, process scheduling, threading, ...	4	DM
Concurrency, race conditions, synchronization	6	DM
Memory management, swapping, virtual memory, ...	6	DM/CC
File Systems, file structures, management, ...	7	CC
Revision	1	CC

Table: Preliminary course structure

Labs

What and How?

- Labs are on **Wednesday 11:00 - 12:00** in A32, CS (from W/C 5th of October)
- The **labs** will teach you:
 - OS concepts (processes, schedulers, shared memory)
 - The use of operating system APIs & implementation / coding on Linux systems
 - The basics of **concurrency**
 - The lab material is examinable
 - The lab is also a good time to check the practical aspects of the coursework - can you log in to the right systems, access the right tools. . .
- **Lectures** will **introduce** the required concepts

Coursework

Content

- The coursework focuses on a **OS structures, process management, concurrency, and threads**
 - **Draft specification** will be available **W/C 12th of October** on Moodle
 - Follow the guidance to **break it down** in steps!
- It requires **C programming**

Submission

- The recommended submission date is **Friday 11th of December**
(latest date is Monday **04/01/2027**)
- NO late submissions (unless you have ECs)!

Reading Material

My Favourite Books

- Seminal books:
 - Tanenbaum, Andrew S. 2014 *Modern Operating Systems*. 4th ed. Prentice Hall Press, Upper Saddle River, NJ, USA.
 - Silberschatz, Abraham, Peter Baer Galvin, Greg Gagne. 2008. *Operating System Concepts*. 8th ed. Wiley Publishing.
 - Stallings, William. 2008. *Operating Systems: Internals and Design Principles*. 6th ed. Prentice Hall Press, Upper Saddle River, NJ, USA.
- Other sources:
 - Daniel P. Bovet, Marco Cesati *Understanding the Linux Kernel*. 3rd ed. O'Reilly Media, November 2005
 - Slides and recordings will be available on Moodle

Assessment

Exam & labs

- **In person closed book ExamSys** (2 hours) on **knowledge, understanding, application**

- The exam counts for 50%. There are four major parts on:

- The introductory material and processes
- Concurrency
- Memory management
- Filesystems

Students answer questions in **3 out of 4** parts.

- **Sample questions** from previous years are available on Moodle and are included in the lectures (**answers** are not available)

- Labs material is **examinable**:

- Some questions in the **exam will be designed to evaluate the labs**
- They also help with some aspects of the coursework

Assessment

- The coursework is an **individual** task and counts for 50%
 - **Git repositories** have been set up for you
 - **Only the final version** should be submitted in Moodle
 - **Submit** your code **regularly** (Git and Moodle- as many times as you like)
- **Academic misconduct** will be followed up on!
 - If you don't understand the instructions **ask the lecturers**
 - If you encounter exceptional circumstances request an EC, **don't help or get help from your peers**
 - **Start early** on the coursework - give yourself time to gradually digest the problem

Assessment

Workload

- This is a **20 credit module** - **200 hours** of work (5×40 hour week)
- The **coursework** should take approximately 100 hours
- **Lectures** take approximately 24 hours, **labs** 9 hours
- 67 hours of **revision**, approx 3 hours per **lecture**

About Us

Contact Details

- DM's Contact details (first half of the course):
 - Name: Dan Marsden
 - Office: C37
 - E-mail: Dan.Marsden@Nottingham.ac.uk
 - Office hour: Thursday 14:00 - 15:00 (**e-mail in advance**)
- GDM's Contact details (second half of the course):
 - Name: Chao Chen
 - E-mail: Chao.Chen@Nottingham.ac.uk
 - Office: B38
 - Office hour: Tuesday 16:00 - 17:00 (**e-mail in advance**)

Quick Lecturer Introduction

Name: Dan Marsden

Interests: Theoretician interested in category theory, logic, and the foundations of computer science, working in the Functional Programming Lab

Quick Lecturer Introduction

Name: Dan Marsden

Interests: Theoretician interested in category theory, logic, and the foundations of computer science, working in the Functional Programming Lab

Previous life: Approximately 12 years working as a professional developer in industry. Coauthor of two libraries in the Boost C++ library project `www.boost.org`

Prerequisites - C Programming

```
#include <stdio.h>
#include <stdlib.h>

#define BUFFER_LENGTH 5

void set(int* value, int const i) {
    *value = i * i;
}

int main() {
    int* const buffer = malloc(BUFFER_LENGTH * sizeof(int));
    if(!buffer) abort();
    for(int i = 0; i != BUFFER_LENGTH; ++i)
        set(buffer + i, i);
    for(int i = 0; i != BUFFER_LENGTH; ++i)
        printf("buffer[%d] = %d\n", i, buffer[i]);
    free(buffer);
}
```

Defining Operating Systems

What Can an OS Do For Me?

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;

public class Demol {
    public static void main(String[] args) throws IOException {
        FileWriter fw =
            new FileWriter(("C:/Program Files (x86)/test.txt"));
        PrintWriter pw = new PrintWriter(fw);
        pw.close();
    }
}
```

Defining Operating Systems

What Can an OS Do For Me?

- **File systems**: **where** is the file physically written on the disk and how is it **retrieved**?
- **Abstraction**: why does the code look the same **independent of the device**?
- **Multi-programming**: what if multiple programs **access the same file simultaneously**?
- **Security**: why might we be **denied access** to the file?

Defining Operating Systems

What Can an OS Do For Me?

```
public class Demo2 {  
    public static void main(String[] args){  
        long[] aLargeArrayOfLargeNumbers  
            = new long[Integer.MAX_VALUE];  
    }  
}
```

virtual memory

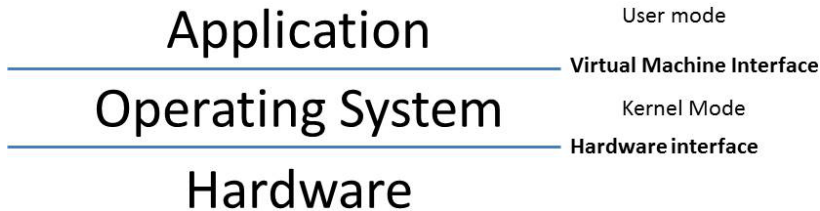
virtual address space

- **Where** in memory will the array be stored and how is it **protected** from unauthorised access?
- What if the array requires **more memory than physically available**? virtual memory
- What if only **part of the array is currently in use** ? demand paging
- What if an **other process starts running**?

Defining Operating Systems

A Virtual Machine Providing Abstractions

- In the early days, programmers had to **deal directly with the hardware**
 - Real computer **hardware is ugly**
 - Hardware is **extremely difficult to manipulate/program**
- An operating system is a layer of indirection on top of the hardware:
 - It provide **abstractions** for application programs (e.g., file systems)
 - It provides a **cleaner and easier interface to the hardware** and hides the complexity of “**bare metal**”
 - It allows the programmer to be lazy by using **common routines :-)**



Defining Operating Systems

Some Wisdom

David Wheeler (First PhD in Computer Science, 1951)

“All problems in computer science can be solved by another level of indirection”

Defining Operating Systems

A Resource Manager

- Many modern operating systems use multi-programming to **improve user experience** and **maximise resource utilisation**
 - Disks are slow: without multi-programming, CPU time is wasted while waiting for I/O requests
 - Imagine a **CPU** running at 3.2 GHz (approx. 3.2×10^9 instructions per second)
 - Imagine a **disk** rotating at 7200 RPM, taking 4.2 ms to rotate half a track
 - I/O is slow, we are **missing out on $3.2 \times 4.2 \times 10^6$ instructions** (13.44m)!
- The implementation of **multi-programming** has important **consequences** for **operating system design**

Defining Operating Systems

A Resource Manager

- The operating system must **allocate/share** resources (including CPU, memory, I/O devices) **fairly** and **safely** between **competing processes**:
 - In time, e.g. CPUs and printers
 - In space, e.g., memory and disks
- The execution of **multiple programs** (processes) needs to be **interleaved** with one another:
 - This requires **context switches** and **process scheduling** $\Rightarrow$ **mutual exclusion, deadlock avoidance, protection, ...**

Summary

Take-Home Message

- An operating system is the fundamental abstraction that lets us safely and efficiently run programs on our hardware.

Summary

Take-Home Message

- An operating system is the fundamental abstraction that lets us safely and efficiently run programs on our hardware.
- Don't forget to brush up your C programming if needed!